Lecture 12 – Examples of Context-Free Grammars

COSE215: Theory of Computation

Jihyeok Park

PLRG

2026 Spring

Recall

- A **context-free grammar (CFG)**:

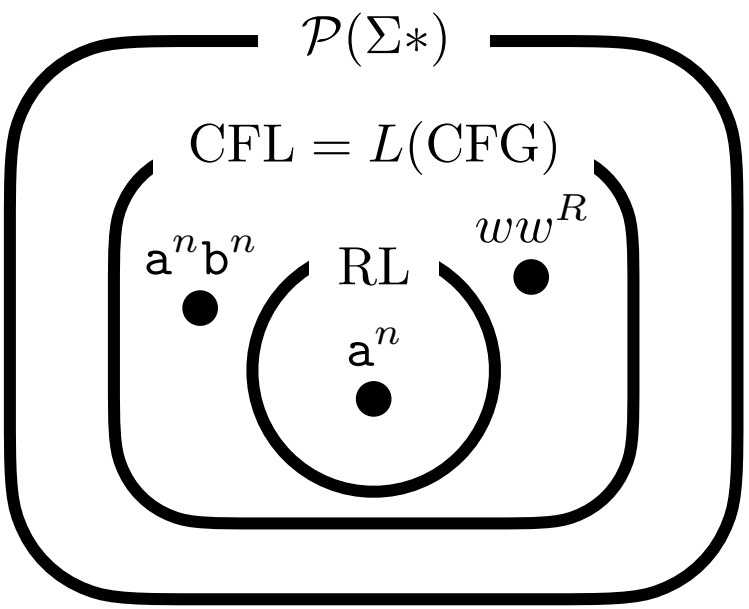
$$G = (V, \Sigma, S, R)$$

- The **language** of a CFG G:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

- A language L is a **context-free language (CFL)**:

$$\exists \text{ CFG } G.\ L(G) = L$$



Contents

1. Regular Languages are Context-Free
   - Regular Expressions to CFGs
   - ϵ-NFA to CFG

2. Examples of Context-Free Grammars
   - Example 1: $\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n}$
   - Example 2: Well-Formed Brackets
   - Example 3: Equal Number of a's and b's
   - Example 4: Unequal Number of a's and b's
   - Example 5: Arithmetic Expressions
   - Example 6: Regular Expressions
   - Example 7: Simplified Scala Syntax

Contents

1. Regular Languages are Context-Free
   - Regular Expressions to CFGs
   - ϵ-NFA to CFG

2. Examples of Context-Free Grammars
   - Example 1: $\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n}$
   - Example 2: Well-Formed Brackets
   - Example 3: Equal Number of a's and b's
   - Example 4: Unequal Number of a's and b's
   - Example 5: Arithmetic Expressions
   - Example 6: Regular Expressions
   - Example 7: Simplified Scala Syntax

Regular Languages are Context-Free

PLRG

Theorem (RLs are CFLs)

All regular languages are context-free.

Proof) There are two ways to prove this theorem:

1. Converting regular expressions to equivalent CFGs
2. Converting ϵ-NFAs to equivalent CFGs

Regular Expressions to CFGs

For a given regular language L, let's construct an equivalent CFG G using the equivalent regular expression R. $L(G) = L(R)$.

RE R	CFG G
$\varnothing$	$S \to S$
ϵ	$S \to \epsilon$
$a \in \Sigma$	$S \to a$
$R_1 \mid R_2$	$S \to S_1 \mid S_2$
$R_1 \cdot R_2$	$S \to S_1 S_2$
R_1^*	$S \to \epsilon \mid S_1 S$
(R_1)	$S \to S_1$

where S_1 and S_2 are start variables of CFGs G_1 and G_2 such that $L(G_1) = L(R_1)$ and $L(G_2) = L(R_2)$, respectively.

Regular Expressions to CFGs – Examples

For a given RE R, construct a CFG G such that $L(G) = L(R)$.

- $R = \epsilon \,|\, \texttt{ab} \,|\, \texttt{ba}$

$$S \to F \mid D \qquad A \to \texttt{a} \qquad C \to AB \qquad E \to \epsilon$$
$$B \to \texttt{b} \qquad D \to BA \qquad F \to E \mid C$$

  Its simplified version:

$$S \to \epsilon \mid \texttt{ab} \mid \texttt{ba}$$

- $R = (\epsilon \,|\, \texttt{a})^*$

$$S \to \epsilon \mid AS \qquad A \to \epsilon \mid \texttt{a}$$

- $R = (0 \,|\, 1(01^*0)^*1)^*$

$$S \to \epsilon \mid AS \qquad A \to 0 \mid 1B1 \qquad C \to 0D0$$
$$B \to \epsilon \mid CB \qquad D \to \epsilon \mid 1D$$

ϵ-NFAs to CFGs

For a given ϵ-NFA $N^\epsilon = (Q, \Sigma, \delta, q_0, F)$, let's construct a CFG G as:

- For each state $q \in Q$ of N^ϵ, introduce a non-terminal A_q.
- For each transition $q \xrightarrow{a} q'$ of N^ϵ, introduce a production rule:

$$A_q \to aA_{q'}$$

- For each ϵ-transition $q \xrightarrow{\epsilon} q'$ of N^ϵ, introduce a production rule:

$$A_q \to A_{q'}$$

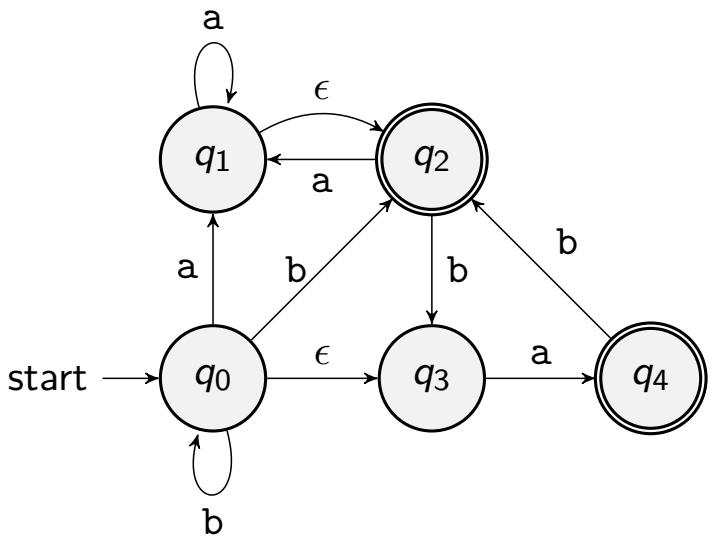
- For each final state $q \in F$ of N^ϵ, introduce a production rule:

$$A_q \to \epsilon$$

- The start variable of G is A_{q_0}.

ϵ-NFAs to CFGs – Examples

$N^\epsilon =$



We can construct a CFG G (A_0 is the start variable) for N^ϵ:

$$
\begin{aligned}
A_0 &\to \mathtt{b}A_0 \mid \mathtt{a}A_1 \mid \mathtt{b}A_2 \mid A_3 \\
A_1 &\to \mathtt{a}A_1 \mid A_2 \\
A_2 &\to \mathtt{a}A_1 \mid \mathtt{b}A_3 \mid \epsilon \\
A_3 &\to \mathtt{a}A_4 \\
A_4 &\to \mathtt{b}A_2 \mid \epsilon
\end{aligned}
$$

For example, we can derive $\mathtt{ba} \in L(N^\epsilon)$ using G:

$$A_0 \Rightarrow \mathtt{b}A_0 \Rightarrow \mathtt{b}A_3 \Rightarrow \mathtt{ba}A_4 \Rightarrow \mathtt{ba}$$

Contents

1. Regular Languages are Context-Free
   - Regular Expressions to CFGs
   - ϵ-NFA to CFG

2. Examples of Context-Free Grammars
   - Example 1: $\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n}$
   - Example 2: Well-Formed Brackets
   - Example 3: Equal Number of a's and b's
   - Example 4: Unequal Number of a's and b's
   - Example 5: Arithmetic Expressions
   - Example 6: Regular Expressions
   - Example 7: Simplified Scala Syntax

Example 1: $\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n}$

Construct a CFG for the language:

$$L = \{\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n} \mid n, m \geq 0\}$$

Let's split a word $w \in L$ using shorter words in L.

$$\forall w \in L.\ w = \begin{cases} ① \ \mathtt{a}^m & \text{for some } m \geq 0 \\ ② \ \mathtt{b}w'\mathtt{bb} & \text{for some } w' \in L \end{cases} \implies S \to A \mid \mathtt{b}S\mathtt{bb}$$

$$\forall m \geq 0.\ \mathtt{a}^m = \begin{cases} ① \ \epsilon \\ ② \ \mathtt{aa}^{m-1} \end{cases} \implies A \to \epsilon \mid \mathtt{a}A$$

Therefore, the following is a CFG for L:

$$\begin{aligned} S &\to A \mid \mathtt{b}S\mathtt{bb} \\ A &\to \epsilon \mid \mathtt{a}A \end{aligned}$$

Example 2: Well-Formed Brackets

Construct a CFG for the language:

$$L = \{w \in \{(,),\{,\},[,]\}^* \mid w \text{ is well-formed}\}$$

Let's split a word $w \in L$ using shorter words in L.

$$\forall w \in L.\ w = \begin{cases} ① \ \epsilon & \\ ② \ (w') & \text{for some } w' \in L \\ ③ \ \{w'\} & \text{for some } w' \in L \\ ④ \ [w'] & \text{for some } w' \in L \\ ⑤ \ w_1 w_2 & \text{for some } w_1, w_2 \in L \end{cases}$$

Therefore, the following is a CFG for L:

$$S \to \epsilon \mid (S) \mid \{S\} \mid [S] \mid SS$$

Example 3: Equal Number of a's and b's

Construct a CFG for the language:

$$L = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid N_{\mathtt{a}}(w) = N_{\mathtt{b}}(w)\}$$

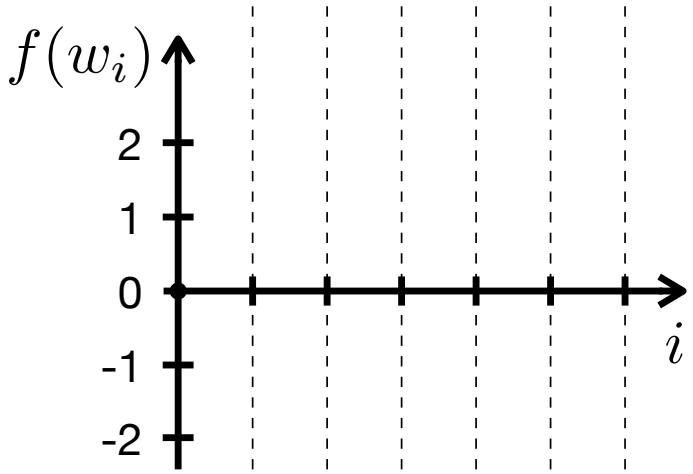
where $N_{\mathtt{a}}(w)$ and $N_{\mathtt{b}}(w)$ are the number of a's and b's in w, respectively.

Consider a function $f(w) = N_{\mathtt{a}}(w) - N_{\mathtt{b}}(w)$.

For example, if $w = \mathtt{abbaaa}$, then $f(w) = N_{\mathtt{a}}(w) - N_{\mathtt{b}}(w) = 4 - 2 = 2$.

If a word $w = a_1 a_2 \cdots a_n \in \{\mathtt{a}, \mathtt{b}\}^*$, let $w_i = a_1 a_2 \cdots a_i$ for $0 \leq i \leq n$.

Let's draw a graph for $f(w_i)$ for $0 \leq i \leq n$:



Example 3: Equal Number of a's and b's

Construct a CFG for the language:

$$L = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid N_{\mathtt{a}}(w) = N_{\mathtt{b}}(w)\}$$

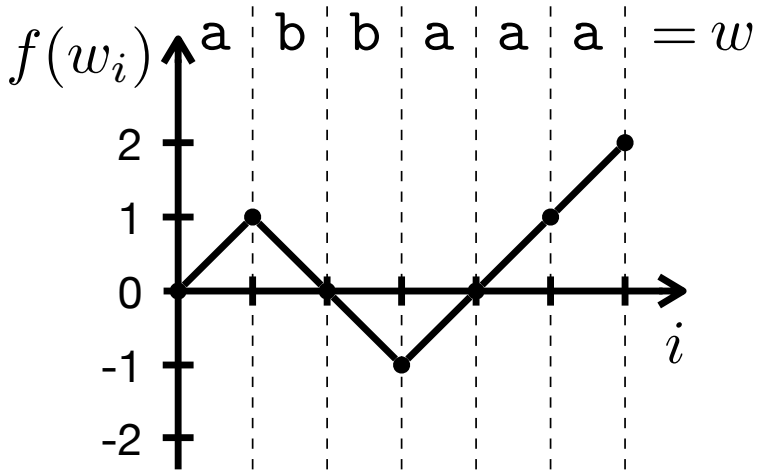
where $N_{\mathtt{a}}(w)$ and $N_{\mathtt{b}}(w)$ are the number of a's and b's in w, respectively.

Consider a function $f(w) = N_{\mathtt{a}}(w) - N_{\mathtt{b}}(w)$.

For example, if $w = \mathtt{abbaaa}$, then $f(w) = N_{\mathtt{a}}(w) - N_{\mathtt{b}}(w) = 4 - 2 = 2$.

If a word $w = a_1 a_2 \cdots a_n \in \{\mathtt{a}, \mathtt{b}\}^*$, let $w_i = a_1 a_2 \cdots a_i$ for $0 \leq i \leq n$.

Let's draw a graph for $f(w_i)$ for $0 \leq i \leq n$:

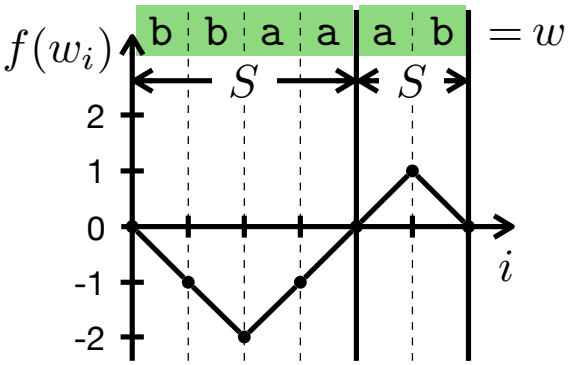


Example 3: Equal Number of a's and b's

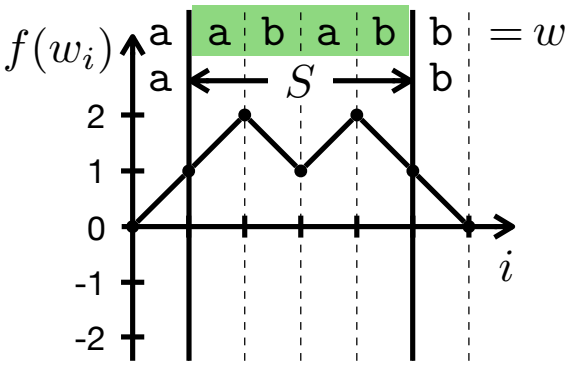
Let's split a word $w \in L$ using shorter words in L.

For a given $w \in L$, there are four cases:

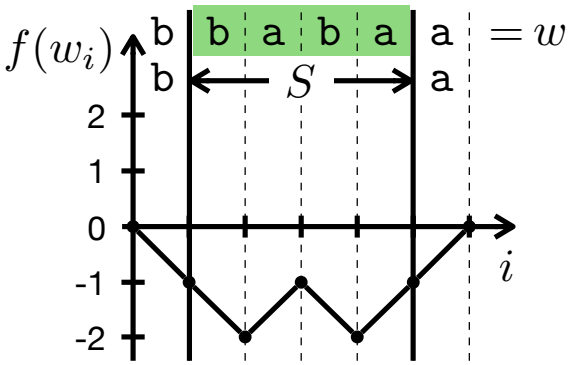
① $w = \epsilon$



② $w = w_1 w_2$



③ $w = \mathtt{a}w'\mathtt{b}$



④ $w = \mathtt{b}w'\mathtt{a}$

Therefore, the following is a CFG for L:

$$S \to \epsilon \mid SS \mid \mathtt{a}S\mathtt{b} \mid \mathtt{b}S\mathtt{a}$$

Example 4: Unequal Number of a's and b's

Construct a CFG for the **complement** of the language in Example 3:

$$L = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid N_{\mathtt{a}}(w) \neq N_{\mathtt{b}}(w)\}$$

where $N_{\mathtt{a}}(w)$ and $N_{\mathtt{b}}(w)$ are the number of a's and b's in w, respectively.

We can categorize $w \in \{\mathtt{a}, \mathtt{b}\}^*$ into three cases using the function f:

- $L_Z = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid f(w) = 0\}$ – equal number of a's and b's
- $L_P = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid f(w) > 0\}$ – more a's than b's
- $L_N = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid f(w) < 0\}$ – more b's than a's

The language L is the disjoint union of L_P and L_N:

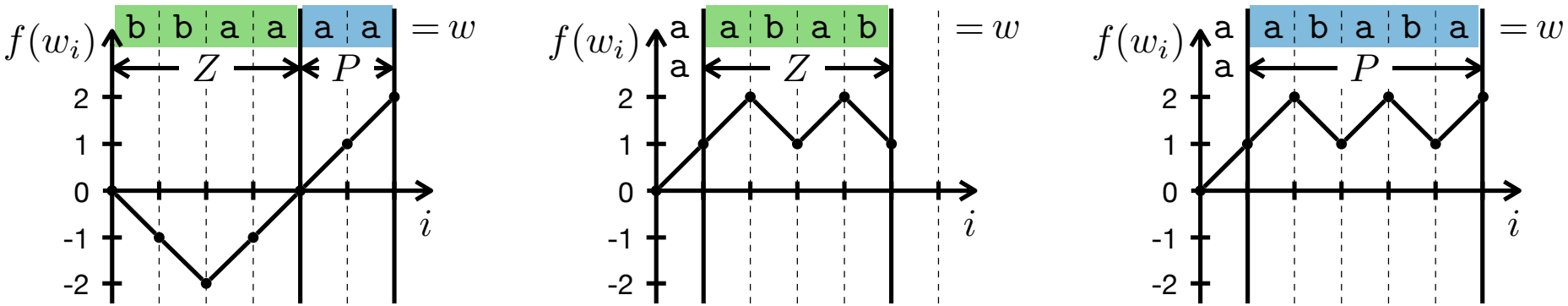
$$L = L_P \uplus L_N$$

Let's define production rules for L_P and L_N using graphs for $f(w_i)$.

Example 4: Unequal Number of a's and b's

Let's split a word $w \in L_P$ using shorter words in L_Z, L_P, and L_N.

For a given $w \in L_P$, there are three cases:



① $w = w_1 w_2$
$(w_1 \in L_Z, w_2 \in L_P)$

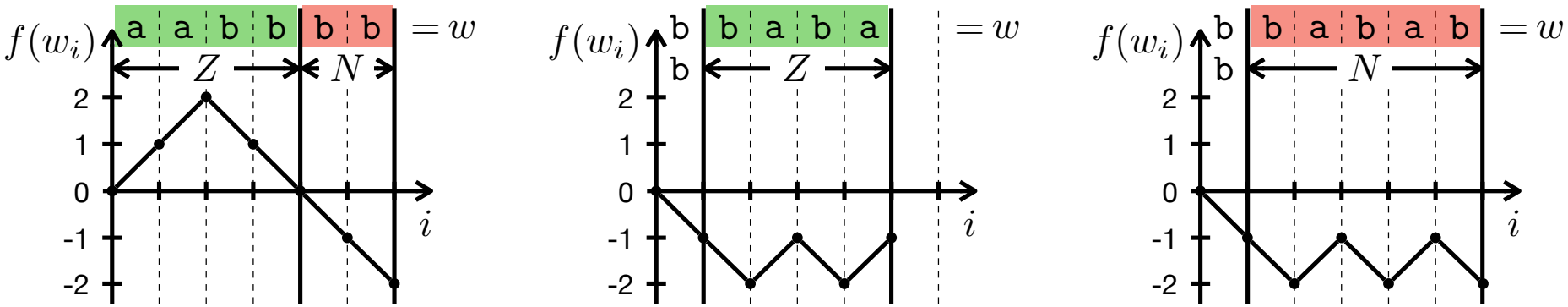
② $w = \mathtt{a}w'$
$(w' \in L_Z)$

③ $w = \mathtt{a}w'$
$(w' \in L_P)$

Therefore, the following is production rules for L_P:

$$P \to ZP \mid \mathtt{a}P \mid \mathtt{a}Z$$

Example 4: Unequal Number of a's and b's



① $w = w_1 w_2$
$(w_1 \in L_Z, w_2 \in L_N)$

② $w = \mathtt{b}w'$
$(w' \in L_Z)$

③ $w = \mathtt{b}w'$
$(w' \in L_N)$

Similarly, the following is production rules for L_N:

$$N \to ZN \mid \mathtt{b}N \mid \mathtt{b}Z$$

Therefore, the CFG for L is:

$$\begin{aligned}
S &\to P \mid N \\
P &\to ZP \mid \mathtt{a}P \mid \mathtt{a}Z \\
N &\to ZN \mid \mathtt{b}N \mid \mathtt{b}Z \\
Z &\to \epsilon \mid ZZ \mid \mathtt{a}Z\mathtt{b} \mid \mathtt{b}Z\mathtt{a}
\end{aligned}$$

Example 5: Arithmetic Expressions

An **arithmetic expression** is defined with the following CFG:

$$
\begin{aligned}
S &\rightarrow N \mid X \mid S{+}S \mid S{*}S \mid (S) \\
N &\rightarrow D \mid DN \\
D &\rightarrow \texttt{0} \mid \cdots \mid \texttt{9} \\
X &\rightarrow \texttt{a} \mid \cdots \mid \texttt{z}
\end{aligned}
$$

We can **derive** an arithmetic expression `13*(2+x)` as follows:

$$
\begin{array}{lllllllll}
S & \Rightarrow & S{*}S & \Rightarrow & N{*}S & \Rightarrow & DN{*}S & \Rightarrow & 1N{*}S \\
 & \Rightarrow & 1D{*}S & \Rightarrow & 13{*}S & \Rightarrow & 13{*}(S) & \Rightarrow & 13{*}(S{+}S) \\
 & \Rightarrow & 13{*}(N{+}S) & \Rightarrow & 13{*}(D{+}S) & \Rightarrow & 13{*}(2{+}S) & \Rightarrow & 13{*}(2{+}X) \\
 & \Rightarrow & 13{*}(2{+}\texttt{x}) & & & & & &
\end{array}
$$

Example 6: Regular Expressions

Consider a language representing the **syntax of regular expressions**:

$$L = \{w \in \{\varnothing, \varepsilon, \mathtt{a}, \mathtt{b}, \mathtt{|}, *, (,)\}^* \mid w \text{ is a regular expression over } \{\mathtt{a}, \mathtt{b}\}\ \}$$

Is this language L **regular**? or **context-free**?

We can prove that L is **not regular** using the pumping lemma.
(Hint: consider a word $({}^n\varepsilon)^n$ for a given $n > 0$)

However, the language L is **context-free**:

$$S \to \varnothing \mid \varepsilon \mid \mathtt{a} \mid \mathtt{b} \mid S\mathtt{|}S \mid SS \mid S* \mid (S)$$

We can **derive** a regular expression $(\mathtt{b}\mathtt{|}\mathtt{ab})*$ as follows:

$$\begin{array}{lllllll} S & \Rightarrow & S* & \Rightarrow & (S)* & \Rightarrow & (S\mathtt{|}S)* \\ & \Rightarrow & (S\mathtt{|}SS)* & \Rightarrow & (S\mathtt{|}S\mathtt{b})* & \Rightarrow & (S\mathtt{|}\mathtt{ab})* \\ & \Rightarrow & (\mathtt{b}\mathtt{|}\mathtt{ab})* & & & & \end{array}$$

Example 7: Simplified Scala Syntax

We can define a CFG for a simplified version of Scala syntax[1]:

(Scala Program)	$S \to E \mid S \ ; \ E$
(Expressions)	$E \to N \mid X \mid E + E \mid E - E \mid E * E \mid E \ / \ E$
	$\mid$ `val` X `:` T `=` E
	$\mid$ `def` X `(` P `):` T `=` E
	$\mid$ E `(` A `)`
	$\mid$ `if` `(` E `)` E `else` E
	$\mid$ `enum` T `{` D `}`
	$\mid$ E `match` `{` C `}`
(Numbers)	$N \to \mathtt{0} \mid \cdots \mid \mathtt{9} \mid \mathtt{0}N \mid \cdots \mid \mathtt{9}N$
(Variables)	$X \to Y \mid YX$
	$Y \to \varnothing \mid \mathtt{a} \mid \cdots \mid \mathtt{z} \mid \mathtt{A} \mid \cdots \mid \mathtt{Z}$
(Types)	$T \to X \mid T \ \mathtt{[} \ T \ \mathtt{]} \mid T \ \mathtt{=>} \ T$
(Parameters)	$P \to \epsilon \mid X \ : \ T \mid P \ , \ X \ : \ T$
(Arguments)	$A \to \epsilon \mid E \mid A \ , \ E$
(Cases)	$C \to$ `case` E `=>` $E \mid C$ `;` `case` E `=>` E
(Enum Cases)	$D \to$ `case` T `(` P `)` $\mid D$ `;` `case` T `(` P `)`

[1] https://docs.scala-lang.org/scala3/reference/syntax.html

Example 7: Simplified Scala Syntax

```
def sum(n: Int): Int = n match { case 0 => 0; case n => n + sum(n - 1) }
```

A derivation for this program:

$S \Rightarrow^*$ `def` X `(` P `):` T `=` E $\Rightarrow^*$ `def sum(` P `):` T `=` E

$\Rightarrow^*$ `def sum(` X `:` T `):` T `=` E $\Rightarrow^*$ `def sum(n: Int): Int =` E

$\Rightarrow^*$ `def sum(n: Int): Int =` E `match {` C `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match {` C `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case` E `=>` E `;` C `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0;` C `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0; case` E `=>` E `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0; case n =>` E `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0; case n =>` E `+` E `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0; case n => n +` E `}`

$\Rightarrow^*$ `def sum(n: Int): Int = n match { case 0 => 0; case n => n + sum(n - 1) }`

Summary

1. Regular Languages are Context-Free
   - Regular Expressions to CFGs
   - ϵ-NFA to CFG

2. Examples of Context-Free Grammars
   - Example 1: $\mathtt{b}^n\mathtt{a}^m\mathtt{b}^{2n}$
   - Example 2: Well-Formed Brackets
   - Example 3: Equal Number of a's and b's
   - Example 4: Unequal Number of a's and b's
   - Example 5: Arithmetic Expressions
   - Example 6: Regular Expressions
   - Example 7: Simplified Scala Syntax

Midterm Exam

- The midterm exam will be given in class.
- **Date:** 13:30-14:45 (1 hour 15 minutes), April 22 (Wed.).
- **Location:**
  - **IT-Education Hall B102:** Students with Student ID 2025XXXX
  - **IT-Education Hall 611:** All other students
- **Coverage:** Lectures 1 – 13
- **Format:** 7–9 questions with closed book and closed notes
- Please refer to the **previous exams** in the course website:

  https://plrg.korea.ac.kr/courses/cose215/

- **Apr. 27 (Mon.):** No class
- **Apr. 29 (Wed.):** Recorded video lecture (Online).

Next Lecture

- Parse Trees and Ambiguity

Jihyeok Park
jihyeok_park@korea.ac.kr
https://plrg.korea.ac.kr